

SML Assignment 3

Q1

Dataset used

Train Images (First 10)



Test Images (First 10)



Implementation

PCA

Principal Component Analysis (PCA) was implemented to reduce the dimensionality of the input data before training. The data was first mean-centered by subtracting the feature-wise mean. The covariance matrix of the centered data was then computed, followed by eigenvalue decomposition to obtain eigenvalues and eigenvectors. The eigenvectors were sorted in descending order of eigenvalues, and the top p eigenvectors were selected to form the projection matrix. The original data was then projected onto this lower-dimensional subspace to obtain reduced representations used for further modeling.

Ridge Regression

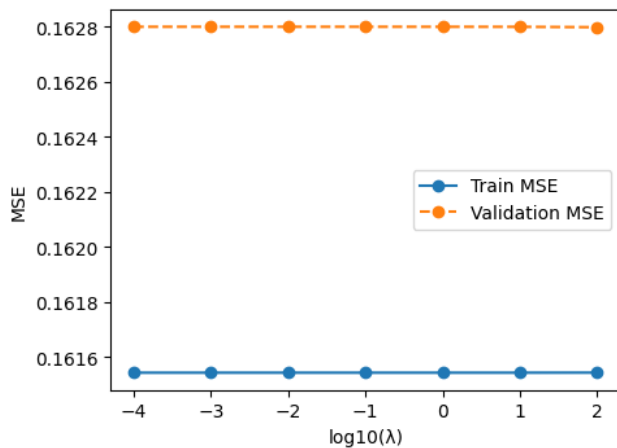
Ridge regression was implemented using the closed-form solution with L2 regularization. For multi-class classification, a one-vs-rest (OVR) strategy was used, where a separate model was trained for each class. The weight vector was computed as $(X^T X + \lambda I)^{-1} X^T y$, where λ controls the strength of regularization. Models were trained across multiple λ values, and both training and validation mean squared error (MSE) were computed. The learned coefficients were also stored to analyze the effect of regularization on parameter shrinkage.

Lasso Regression

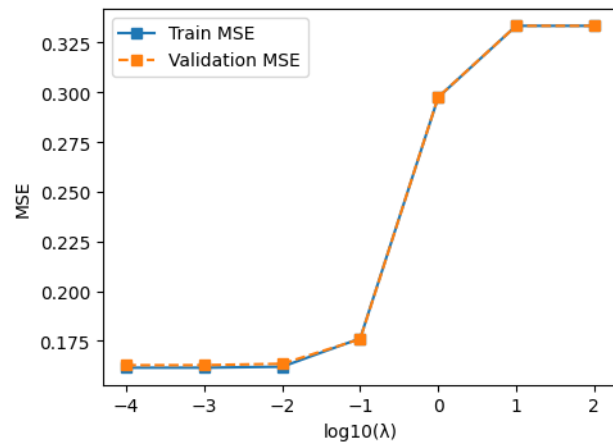
Lasso regression was implemented with L1 regularization to promote sparsity in model coefficients. Since Lasso does not have a closed-form solution, a coordinate descent-based implementation from a standard library was used. Similar to Ridge, an OVR approach was followed for multi-class classification. Models were trained for different values of λ , and training and validation MSE were evaluated. Additionally, the number of non-zero coefficients was tracked to study sparsity patterns and the impact of regularization.

Experiment 1: Train models for different values of λ

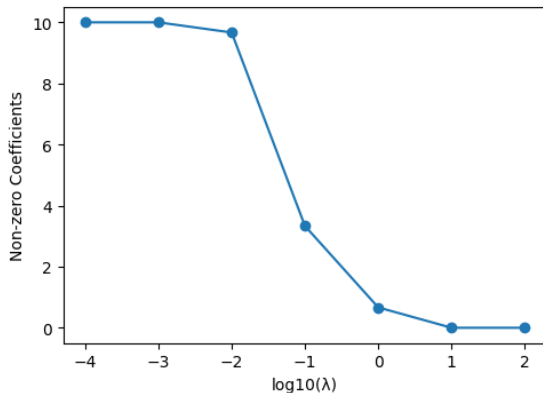
Ridge Regression: Error(MSE) vs Regularization Strength (λ)



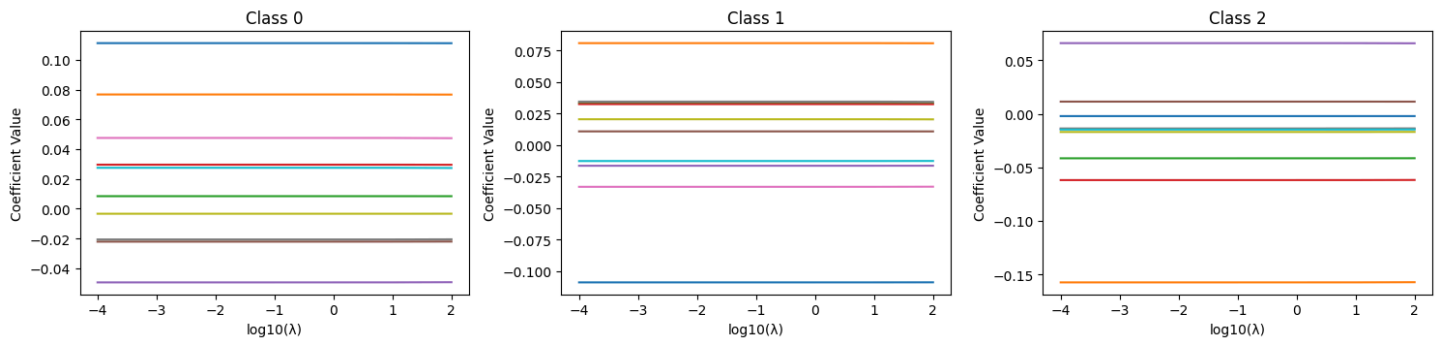
Lasso Regression: Error(MSE) vs Regularization Strength (λ)



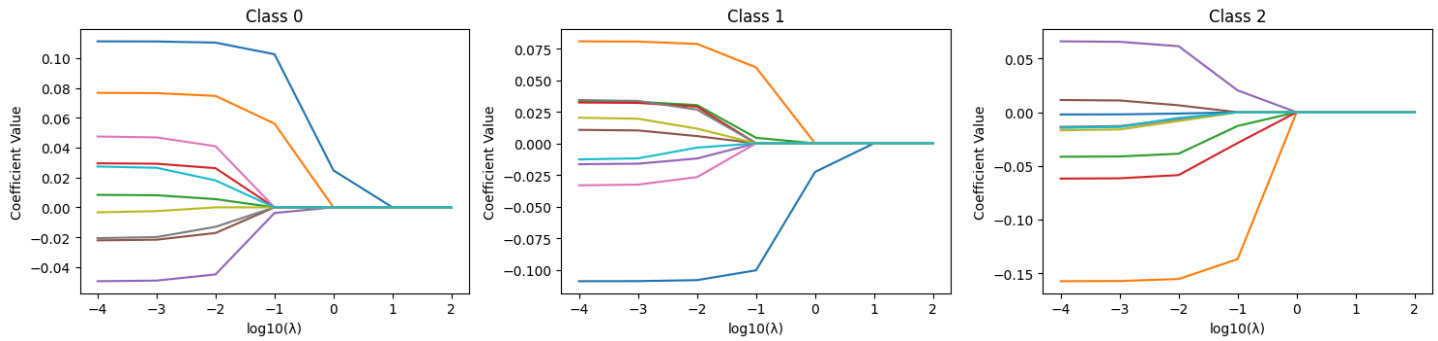
Lasso Regression: Sparsity vs Regularization Strength (λ)



Ridge Regression: Coefficient Paths vs Regularization Strength (λ)



Lasso Regression: Coefficient Paths vs Regularization Strength (λ)



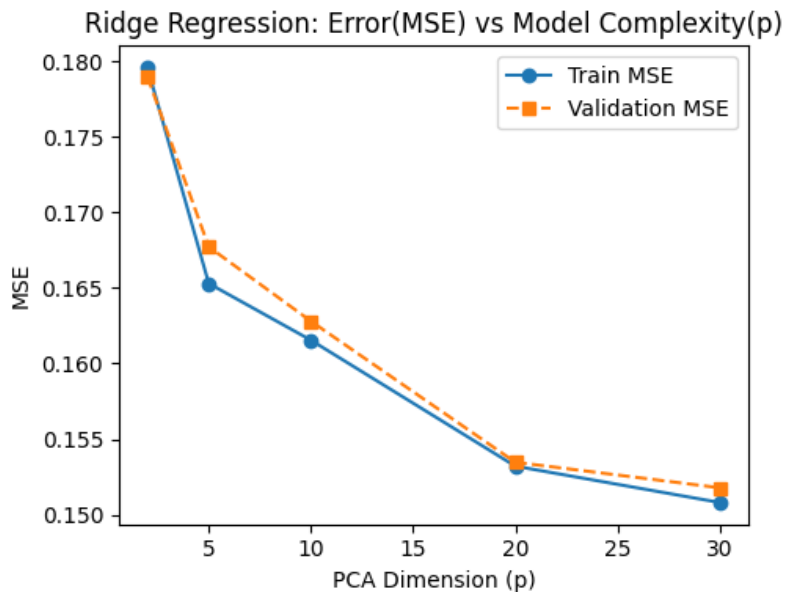
Best parameter:

Ridge: $\lambda = 100$, Avg Val MSE= 0.16280

Lasso: $\lambda = 0.0001$, Avg Val MSE= 0.16280

Experiment2: Train Ridge models for different model complexity (p) with best λ

$\lambda = 100$ was used



Experiment3: Choose best model and test performance

Best Model Overall: Ridge with $\lambda = 100$

Final Test Accuracy (Ridge): 0.9514

Final Analysis

After PCA reduces the data to 10 orthogonal components, Ridge regression shows almost no change in training and validation MSE across different λ values because the features are already uncorrelated and the system is well-conditioned, so regularization has little effect; this is also reflected in nearly flat coefficient paths.

In contrast, Lasso regression is highly sensitive to λ , as increasing regularization forces many coefficients to exactly zero, leading to loss of important information and a sharp increase in error.

The coefficient paths confirm this behavior, with Ridge smoothly shrinking weights while Lasso performs aggressive feature selection.

As model complexity (p) increases, there are more components for the model to learn, hence both training and validation errors decrease, indicating improved representation capacity without clear overfitting in the tested range. Overall, Ridge performs more stably and is better suited for this PCA-transformed setting, while Lasso can degrade performance due to excessive sparsity.

Q2

Dataset used

Train Images (First 10)



Test Images (First 10)



Implementation

PCA

Principal Component Analysis (PCA) was implemented to reduce the dimensionality of the input data before training. The data was first mean-centered by subtracting the feature-wise mean. The covariance matrix of the centered data was then computed, followed by eigenvalue decomposition to obtain eigenvalues and eigenvectors. The eigenvectors were sorted in descending order of eigenvalues, and the top p eigenvectors were selected to form the projection matrix. The original data was then projected onto this lower-dimensional subspace to obtain reduced representations used for further modeling.

Decision Tree

A decision tree classifier was implemented using the Weighted Gini Index as the splitting criterion. For each node, all (p) features were considered (in the case of a standard decision tree), and candidate thresholds were chosen as the mean and median of each feature. For every feature-threshold pair, the dataset was split into left and right regions, and the weighted Gini impurity was computed. The split that minimized the weighted Gini index was selected.

The tree was grown to have exactly three terminal nodes. First, an optimal root split was performed on the entire dataset. Then, both resulting child nodes were evaluated for further splitting, and the node with the highest reduction in Gini impurity was selected for the second split. This resulted in partitioning the feature space into three regions. Each leaf node was assigned a class label based on the majority class of training samples within that region. Predictions for test samples were obtained by traversing the tree based on the learned feature thresholds.

Bagging and Out-of-Bag (OOB) Error

Bagging was implemented by training multiple decision trees on bootstrap samples of the training data. Each bootstrap dataset was created by sampling with replacement and had the same size as the original training set. For each tree, the samples not included in the bootstrap dataset were treated as Out-of-Bag (OOB) samples and used to estimate the generalization error.

Each tree was trained independently using the same decision tree procedure. The OOB error was computed by aggregating misclassification errors across all trees and normalizing by the total number of OOB samples. For prediction, each tree produced a class label for the test data, and final predictions were obtained using majority voting across all trees. This ensemble approach helps reduce variance and improves robustness compared to a single decision tree.

Random Forest

Random Forest was implemented by extending the bagging approach with additional feature randomness. In addition to bootstrap sampling, at each split of the decision tree, only a random subset of k features ($k=\sqrt{p}$) was considered for selecting the best split. This reduces correlation between trees and improves generalization performance.

The same tree structure (with a maximum of three terminal nodes) was used. Trees were trained on bootstrap datasets, and OOB error was computed similarly to bagging. Final predictions were obtained through majority voting across all trees. The performance of Random Forest was compared with bagging to analyze the effect of feature randomness on model accuracy and generalization.

PCA dimension of train data: 10

PCA dimension of test data: 10

Experiment 1: Decision Tree

We use the mean/median of each dimension as the split threshold.

Since PCA components are centered at zero and represent directions of maximum variance, mean/median serves as a statistically sound central pivot for normalized data.

Mode is not considered since PCA-transformed features are continuous and rarely have repeated values, making the mode unstable and not meaningful as a split threshold.

Tree Info:

Root: feature 0 | median = -0.4768

Right split: feature 1 | mean = -0.3252

Single Decision Tree:

Class 0 Accuracy: 0.8755

Class 1 Accuracy: 0.9974

Class 2 Accuracy: 0.5329

Overall Accuracy: 0.8071

Experiment 2: Bagging

Weighted OOB Error (Bagging): 0.1904

Bagging Ensemble:

Class 0 Accuracy: 0.8755

Class 1 Accuracy: 0.9974

Class 2 Accuracy: 0.5271

Overall Accuracy: 0.8052

Experiment 3: Random Forest

Weighted OOB Error (Random Forest): 0.3127

Random Forest:

Class 0 Accuracy: 0.8184

Class 1 Accuracy: 0.7568

Class 2 Accuracy: 0.9196

Overall Accuracy: 0.8294

The number of features considered at each split was chosen as $k = \sqrt{p}$. For $p = 10$, this gives $k \approx 3$.

This choice follows a standard heuristic used in Random Forests, where selecting $\sqrt{p}$ features helps balance the trade-off between tree diversity and predictive strength. By using a smaller subset of features at each split, the correlation between trees is reduced, which improves the overall performance of the ensemble. At the same time, enough information is retained to make meaningful splits.

Empirically, this value of k provided good performance in terms of both test accuracy and OOB error.

Reference: [Breiman, L. \(2001\). Random Forests. Machine Learning, 45\(1\), 5–32.](#)

Final Analysis

YES, this approach is better.

Random Forest performed slightly better than bagging in terms of overall test accuracy, mainly because random feature selection decorrelates the trees and improves generalization.

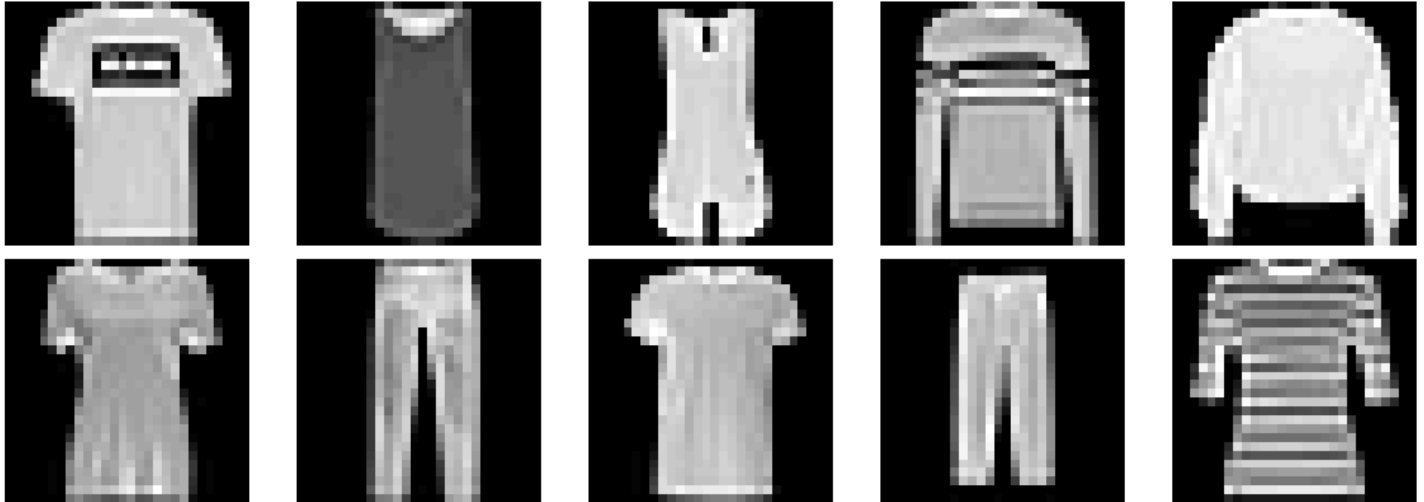
However, the benefit was not consistent across classes, and the OOB error was higher, indicating that the improvement was limited and class-dependent.

Additionally, since PCA reduces the data to a low-dimensional (784 to 10) and uncorrelated feature space, the advantage of feature subsampling in Random Forest is reduced. This may limit its performance gains compared to bagging, which utilizes all available features at each split.

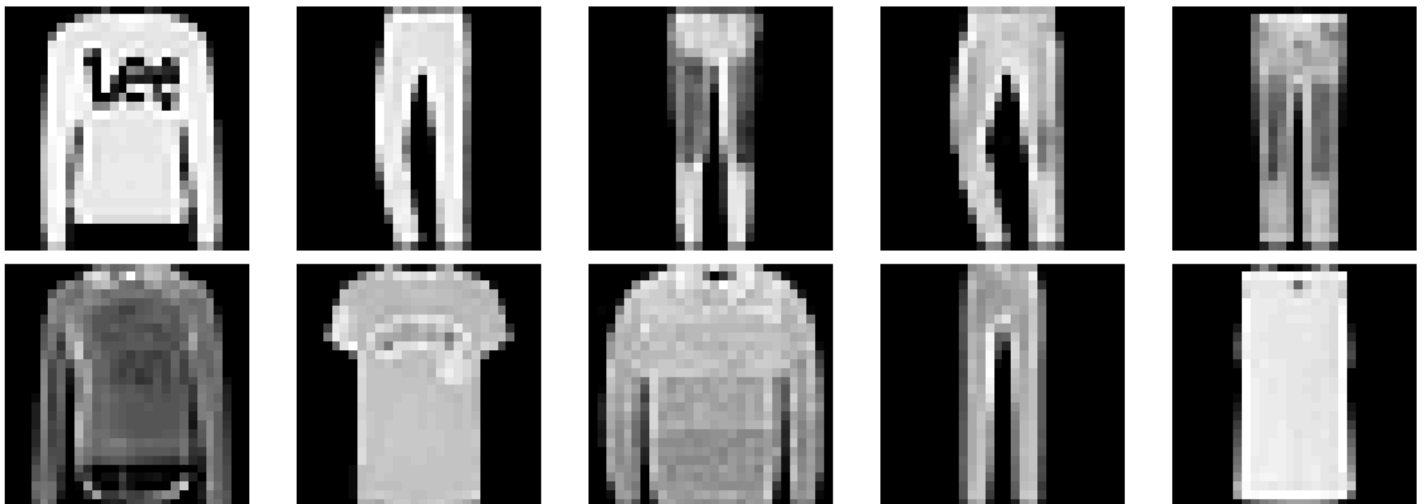
Q3

Dataset used

Train Images (First 10)



Test Images (First 10)



Implementation

PCA

Principal Component Analysis (PCA) was implemented to reduce the dimensionality of the input data before training. The data was first mean-centered by subtracting the feature-wise mean. The covariance matrix of the centered data was then computed, followed by eigenvalue decomposition to obtain eigenvalues and eigenvectors. The eigenvectors were sorted in descending order of eigenvalues, and the top p eigenvectors were selected to form the projection matrix. The original data was then projected onto this lower-dimensional subspace to obtain reduced representations used for further modeling.

Decision Stump (Regression)

A regression decision stump (a decision tree with a single split) was implemented. For each feature, the values were sorted in ascending order, and candidate split thresholds were chosen as the midpoints between consecutive sorted values. For every candidate threshold, the dataset was divided into left and right regions.

The quality of each split was evaluated using the Sum of Squared Residuals (SSR), computed as the sum of squared differences between the target values and the mean response in each region. The split that minimized SSR was selected as the optimal split. The model stores the selected feature, threshold, and the mean response values for the left and right regions.

Bagging and Out-of-Bag (OOB) Error

Bagging was implemented by training multiple decision stumps on bootstrap samples of the training data. Each bootstrap dataset was created by sampling with replacement and had the same size as the original dataset. For each model, the samples not included in the bootstrap dataset were treated as Out-of-Bag (OOB) samples.

The OOB error was computed using Mean Squared Error by evaluating each model on its corresponding OOB samples. The average OOB error across all models was used as an estimate of generalization performance. For prediction, outputs from all models were averaged to produce the final prediction. This aggregation reduces variance and improves the stability of the model compared to a single decision stump.

PCA dimension of train data: 10

PCA dimension of test data: 10

Experiment 1: Decision Stump

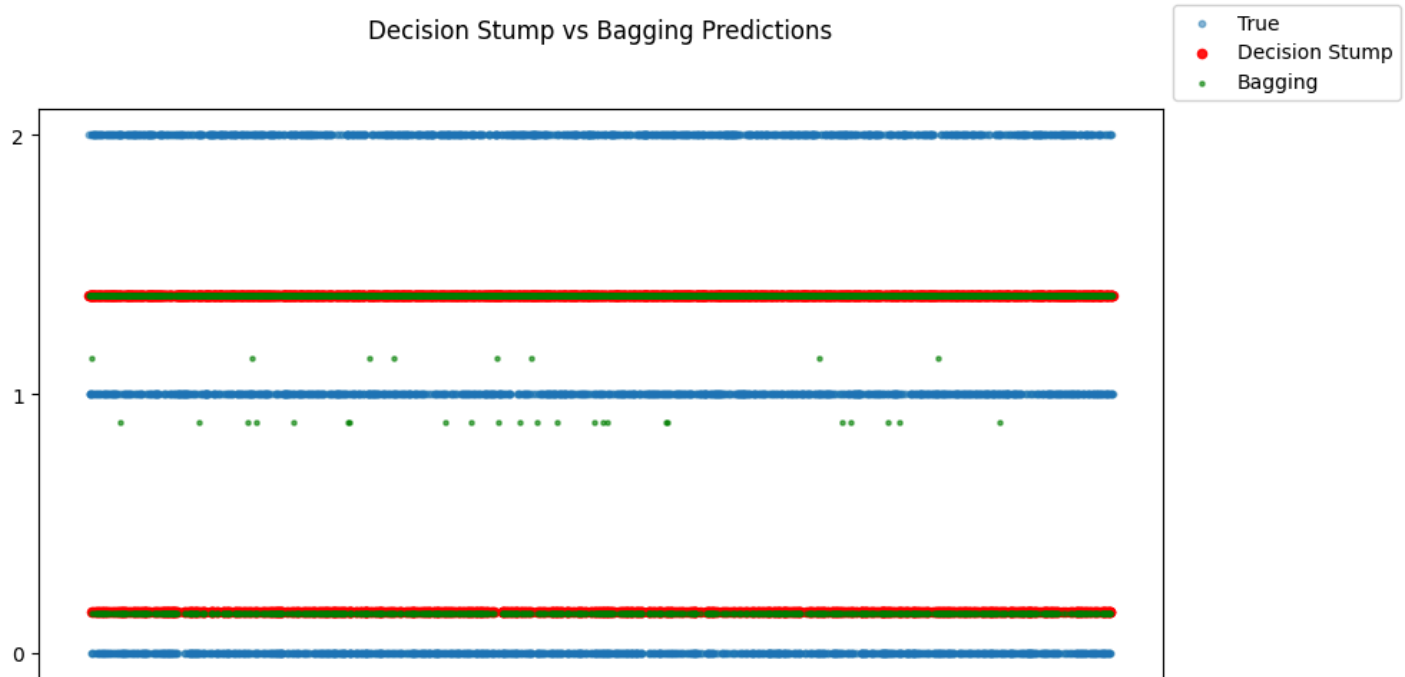
Best Stump: feature 2 | threshold 1.4508

Test MSE (Single Stump): 0.3629

Experiment 2: Bagging

OOB MSE (Bagging): 0.3446

Test MSE (Bagging): 0.3588



The single decision stump produces piecewise constant predictions with only two distinct values, making it overly simple and unable to capture data variability, leading to higher bias.

In contrast, bagging combines multiple stumps trained on different bootstrap samples and averages their predictions. This results in smoother outputs and reduces variance, leading to more stable and accurate predictions.

Final Analysis

Bagging provides a slight improvement over the single decision stump. The test MSE decreases from 0.3629 (stump) to 0.3588 (bagging) and the OOB MSE of 0.3446 indicates better generalization.

The improvement is moderate because the base model (decision stump) is very simple and has high bias, limiting how much performance can be gained even after averaging. Additionally, since PCA reduces the data to a compact and uncorrelated feature space (784 to 10), each component already captures significant information. Using only a single feature for splitting restricts model expressiveness, which further limits the gains from bagging.

Overall, bagging improves stability and generalization by reducing variance, but the simplicity of the underlying model constrains the extent of improvement.